

ISET Sidi Bouzid

Développement Mobile Android avec Java

Cours

02 – LES RESSOURCES

Enseignant : Kheireddine Derbali
Année universitaire : 2026/2027
Département : Département Informatique

Année universitaire 2026/2027

1. L'architecture du dossier /res

Toutes les ressources d'une application Android sont stockées dans le répertoire `res/`. Ce dossier est subdivisé en sous-dossiers spécifiques au type de ressource qu'ils contiennent.

1.1. RES/LAYOUT

Contient les fichiers XML définissant l'interface utilisateur (UI). Chaque fichier correspond généralement à l'apparence d'une Activity, d'un Fragment ou d'un élément de liste réutilisable.

1.2. RES/DRAWABLE/

Contient les éléments graphiques : images bitmap (PNG, JPEG), graphiques vectoriels (XML), et des formes définies en XML (boutons arrondis, dégradés).

1.3. RES/VALUES/

Contient des fichiers XML définissant des valeurs simples : chaînes de caractères (strings), couleurs, dimensions, et styles. (Ex : `strings.xml`).

1.4. RES/MIPMAP/

Exclusivement réservé aux icônes de lancement de l'application (launcher icons). Le système choisit la résolution adaptée à l'écran de l'appareil.

2. La classe R (R.java)

Lors de la compilation, Android génère automatiquement une classe appelée `R`. Cette classe agit comme un annuaire contenant des identifiants entiers uniques pour chaque ressource du dossier `res/`.

NB

Sans la classe `R`, le code Java/Kotlin ne pourrait pas communiquer avec les fichiers XML.

3. Les fichiers du dossier /values

Le dossier `values/` est essentiel pour centraliser les données de l'application. Plutôt que de coder en dur ("hardcoder") des textes ou des couleurs, on y fait référence par des variables.

FICHIER STANDARD	BALISE XML	USAGE PRINCIPAL
<code>strings.xml</code>	<code><string></code>	Tous les textes affichés à l'utilisateur. Indispensable pour faciliter la traduction et la prise en charge de plusieurs langues.
<code>colors.xml</code>	<code><color></code>	Définition de la palette de couleurs de l'application, notamment avec des codes hexadécimaux comme <code>#AARRGGBB</code> .
<code>dimens.xml</code>	<code><dimen></code>	Définition des dimensions utilisées dans l'interface : tailles de texte en <code>sp</code> , marges, espacements et dimensions des composants en <code>dp</code> .
<code>styles.xml</code> / <code>themes.xml</code>	<code><style></code>	Définition de l'apparence globale de l'application ou personnalisation de l'apparence d'un composant spécifique.

4. Accéder aux ressources (Syntaxe)

La manière d'appeler une ressource diffère selon que l'on se trouve dans un fichier XML ou dans le code Java

Accès depuis un fichier XML

Format général :

`@type_ressource/nom_ressource`

```

1      <!-- Exemple dans activity_main.xml -->
2      <TextView
3          android:layout_width="match_parent" android:layout_height="wrap_content"
4          android:text="@string/titre_bienvenue" android:textColor="@color/noir_profond"
5          android:layout_margin="@dimen/marge_standard"
6      />

```

Accès depuis le code Java

Format général :

`R.type_ressource.nom_ressource`

```

1      String monTexte = getString(R.string.titre_bienvenue);
2
3      Button monBouton = findViewById(R.id.bouton_valider);
4
5      TextView monTextView = findViewById(R.id.mon_texte);

```

5. Qualificateurs et Ressources Alternatives

Android permet de fournir différentes versions d'une même ressource pour s'adapter à la configuration de l'appareil (langue, taille d'écran, orientation, mode sombre). Cela se fait en ajoutant des qualificateurs au nom du dossier.

Appareil	Dossier vérifié	Résultat
Configuré en français	<code>values-fr/</code> (Français)	Sélectionné
Autre langue (non trouvée)	<code>values/</code> (Défaut – Anglais)	Utilisé par défaut

5.1. Exemples de Qualificateurs Courants

- **Langue (Localisation)** : `values-fr` (Français), `values-es` (Espagnol). Si la langue n'est pas trouvée, Android charge le dossier `values/` par défaut.
- **Orientation** : `layout-land` (Paysage), `layout-port` (Portrait).
- **Taille d'écran** : `layout-sw600dp` (Tablettes avec une largeur minimale de 600dp).
- **Mode Sombre** : `values-night` (Valeurs spécifiques quand le mode sombre est activé).

6. Bonnes Pratiques

Bonnes pratiques

- **Ne jamais coder de chaînes en dur** : Toute chaîne de texte destinée à l'utilisateur doit être dans `strings.xml` . Un texte dans le code Java (ex : `"Bonjour"`) générera un avertissement (Warning) du compilateur car il est intraitable pour la traduction.
- **Unité dp pour les dimensions** : Utilisez toujours le Density-independent Pixel (`dp`) pour les tailles et marges afin d'assurer un rendu uniforme sur des écrans de densités différentes.
- **Unité sp pour les textes** : Utilisez le Scale-independent Pixel (`sp`) pour les polices. Contrairement au `dp` , le `sp` s'adapte aux préférences d'accessibilité de l'utilisateur.
- **Images vectorielles en priorité** : Préférez les VectorDrawables (XML) aux PNG. Ils prennent moins d'espace et ne perdent jamais en qualité lors du redimensionnement.